

AI Pathfinder - Comprehensive Report

1. Overview

This report documents the implementation and behavior of six search strategies in the AI Pathfinder project:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform-Cost Search (UCS)
- Depth-Limited Search (DLS)
- Iterative Deepening DFS (IDDFS)
- Bidirectional Search

Each algorithm is evaluated with:

- Best Case Scenario: the target is reached quickly.
- Worst Case Scenario: the layout forces broader/deeper exploration.

All tests use a 10x10 grid with 8-directional movement.

2. Common Implementation Design

- Grid is represented as a matrix (0 empty, -1 wall).
- Start cell is blue, target cell is green.
- Frontier cells are yellow, explored cells are light blue, and final path cells are purple.
- For each run, parent pointers are used to reconstruct the final path.

3. Breadth-First Search (BFS)

Explanation

BFS uses a FIFO queue and explores nodes level-by-level from the start node. Because each move has unit cost, BFS guarantees the shortest path (minimum number of steps) when a path exists.

Pros & Cons

Pros: - Complete and optimal under unit step cost. - Easy to reason about and debug.

Cons: - Can consume significant memory as frontier width grows. - In open areas, it may expand many unnecessary nodes.

Test Metrics

Case	Found	Path Length	Expanded Nodes	Peak Frontier	Runtime (ms)
Best	Yes	6	62	17	0.502
Worst	Yes	10	77	15	0.626

Visual Proof

[illegible]

Worst Case - Initial

BFS - Worst Case (Initial)

0	0	0	0	0	0	0	0	0	0	0
0			0	0	0	0	0	0	0	0
0	-1	-1	-1	0	0	-1	-1	-1	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	-1	-1	0	0	-1	-1	-1	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	-1	-1	0	0	-1	-1	-1	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	-1	-1	0	0	-1	-1	-1	0	0
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0		0	0
0	0	0	0	0	0	0	0		0	0
0	0	0	0	0	0	0	0	0	0	0

Worst Case - Final

BFS - Worst Case (Final)

0	0	0	0	0	0	0	0	0	0	0
0			0	0	0	0	0	0	0	0
0	-1	-1	-1	0	0	-1	-1	-1	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	-1	-1	0	0	-1	-1	-1	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	-1	-1	0	0	-1	-1	-1	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	-1	-1	0	0	-1	-1	-1	0	0
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0

4. Depth-First Search (DFS)

Explanation

DFS uses a LIFO stack and follows one branch deeply before backtracking. It can find a solution quickly in favorable branch ordering but does not guarantee shortest paths.

Pros & Cons

Pros: - Lower memory use than BFS/UCS in many cases. - Can be fast when the first deep branch is near the goal.

Cons: - Not optimal for shortest path. - Highly sensitive to neighbor ordering and maze structure.

Test Metrics

Case	Found	Path Length	Expanded Nodes	Peak Frontier	Runtime (ms)
Best	Yes	44	53	48	0.676
Worst	Yes	14	65	33	0.687

Visual Proof

Best Case - Initial

DFS - Best Case (Initial)									
0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Best Case - Final

DFS - Best Case (Final)									
0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Worst Case - Initial

DFS - Worst Case (Initial)

0	0	0	0	0	0	0	0	0	0	0
0										
0	-1	-1	-1	0	0	-1	-1	-1	0	
0	0	0	0	0	0	0	0	0	0	
0	-1	-1	-1	0	0	-1	-1	-1	0	
0	0	0	0	0	0	0	0	0	0	
0	-1	-1	-1	0	0	-1	-1	-1	0	
0	0	0	0	0	0	0	0	0	0	
0	-1	-1	-1	0	0	-1	-1	-1	0	
0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0			
0	0	0	0	0	0	0	0	0	0	

Worst Case - Final

DFS - Worst Case (Final)

0	0	0	0	0	0	0	0	0	0	0
0										
0	-1	-1	-1	0	0	-1	-1	-1	0	
0	0	0	0	0	0	0	0	0	0	
0	-1	-1	-1	0	0	-1	-1	-1	0	
0	0	0	0	0	0	0	0	0	0	
0	-1	-1	-1	0	0	-1	-1	-1	0	
0	0	0	0	0	0	0	0	0	0	
0	-1	-1	-1	0	0	-1	-1	-1	0	
0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0			
0	0	0	0	0	0	0	0	0	0	

5. Uniform-Cost Search (UCS)

Explanation

UCS uses a priority queue and always expands the lowest cumulative cost node first. With uniform step costs in this grid, UCS behavior is close to BFS while keeping support for weighted environments.

Pros & Cons

Pros: - Complete and optimal for positive costs. - Extends naturally to weighted pathfinding.

Cons: - Priority-queue maintenance adds overhead. - Can still expand many nodes in complex obstacle layouts.

Test Metrics

Case	Found	Path Length	Expanded Nodes	Peak Frontier	Runtime (ms)
Best	Yes	7	77	19	0.992
Worst	Yes	10	76	15	1.408

Visual Proof

Best Case - Initial

UCS - Best Case (Initial)									
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Best Case - Final

UCS - Best Case (Final)									
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Worst Case - Initial

UCS - Worst Case (Initial)										
0	0	0	0	0	0	0	0	0	0	0
0		0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0		0	
0	0	0	0	0	0	0	0	0	0	0

Worst Case - Final

UCS - Worst Case (Final)										
0	0	0	0	0	0	0	0	0	0	0
0		0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0		0	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0

6. Depth-Limited Search (DLS)

Explanation

DLS is DFS with a fixed depth cutoff. Nodes beyond the cutoff are not explored. This keeps depth bounded but can fail to find a valid path if the solution is deeper than the chosen limit.

Pros & Cons

Pros: - Very memory efficient. - Useful when a realistic depth bound is known.

Cons: - Incomplete if depth limit is too low. - Still sensitive to branch ordering like DFS.

Test Metrics

Case	Found	Path Length	Expanded Nodes	Peak Frontier	Runtime (ms)	Notes
Best	Yes	1	25	14	0.287	depth_limit=5
Worst	No	0	74	22	0.952	depth_limit=12

Visual Proof

Best Case - Initial

Best Case - Final

Worst Case - Initial

DLS - Worst Case (Initial)										
0	0	0	0	0	0	0	0	0	0	0
0		0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0		0	
0	0	0	0	0	0	0	0	0	0	0

Worst Case - Final

DLS - Worst Case (Final)										
0	0	0	0	0	0	0	0	0	0	0
0		0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0		0	
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0		0	
0	0	0	0	0	0	0	0	0	0	0

7. Iterative Deepening DFS (IDDFS)

Explanation

IDDFS repeatedly runs depth-limited DFS from depth 0 to `max_depth` until the target is found. It combines DFS-like memory usage with improved completeness for shallow goals.

Pros & Cons

Pros: - Low memory footprint like DFS. - More reliable than single-limit DLS when target depth is unknown.

Cons: - Re-expands upper levels repeatedly, increasing runtime. - Needs a practical maximum depth.

Test Metrics

Case	Found	Path Length	Expanded Nodes	Peak Frontier	Runtime (ms)	Notes
Best	Yes	1	3	8	0.270	max_depth=5, found_depth=1
Worst	Yes	11	428	22	5.005	max_depth=15, found_depth=11

Visual Proof

Best Case - Initial

Best Case - Final

Worst Case - Initial

IDDFS - Worst Case (Initial)

0	0	0	0	0	0	0	0	0	0	0
0		0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	
0	0	0	0	0	0	0	0		0	
0	0	0	0	0	0	0	0	0	0	0

Worst Case - Final

IDDFS - Worst Case (Final)

0	0	0	0	0	0	0	0	0	0	0
0		0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	-1	0	-1	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	-1	0	-1	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	-1	0	-1	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	-1	0	-1	0
0	0	0	0	0	0	0	0	0		0
0	0	0	0	0	0	0	0	0	0	0

8. Bidirectional Search

Explanation

Bidirectional Search runs simultaneous searches from start and target. When frontiers intersect, the two partial paths are merged into a complete path. This often reduces search depth and node expansions.

Pros & Cons

Pros: - Typically explores fewer nodes than one-directional search in large spaces.
- Can reduce effective search depth.

Cons: - More complex implementation (two frontiers and merge logic). - Benefit depends on map symmetry and obstacle distribution.

Test Metrics

Case	Found	Path Length	Expanded Nodes	Peak Frontier	Runtime (ms)
Best	Yes	1	2	2	0.146
Worst	Yes	11	58	17	1.987

Visual Proof

Best Case - Initial

Bidirectional_Search - Best Case (Initial)									
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Best Case - Final

Bidirectional_Search - Best Case (Final)									
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Worst Case - Initial

Bidirectional_Search - Worst Case (Initial)

0	0	0	0	0	0	0	0	0	0	0
0		0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	0	-1	0	0
0	0	0	0	0	0	0	0		0	0
0	0	0	0	0	0	0	0	0	0	0

Worst Case - Final

Bidirectional_Search - Worst Case (Final)

0	0	0	0	0	0	0	0	0	0	0
0		0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	-1	0	-1	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	-1	0	-1	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	-1	0	-1	0
0	0	0	0	0	0	0	0	0	0	0
0	-1	0	-1	-1	-1	-1	-1	0	-1	0
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0

9. Conclusion

- BFS and UCS are most reliable for shortest-path behavior on this grid model.
- DFS and DLS can be efficient in favorable layouts but are less consistent.
- IDDFS improves robustness over DLS but at runtime cost from repeated iterations.
- Bidirectional Search showed strong performance in this test set by reducing total exploration.